

Docker

Same environment everywhere

Last week

You now know how to put code code on GitHub. Someone else can now `git clone` it.

And then it doesn't run on their machine.

- Different Python version
- Missing system library
- A package that only builds on Linux
- "Works for me"

Git versions your **code**. It does not version your **environment**.

Today

Part 1: what a container is, and running other people's containers

Part 2: building your own image with a Dockerfile

Part 3: data, config and multiple containers (Compose)

The problem

- "It works on my machine"
- Different package versions across the team
- Hours setting up an environment for a new team member
- Code breaks moving from development to production
- CI fails for reasons that have nothing to do with your change

Why this happens

- **Operating system differences:** Windows, macOS, Linux
- **Different package managers and versions:** pip, conda, apt, brew
- **Missing system dependencies:** the C library that `pandas` needs
- **Environment variables and configuration:** set once, months ago, and forgotten
- **Hidden dependencies:** things you installed and never wrote down

The environment lives in your your laptop, not in the repo.

Solution: Package the entire environment along with the code.

The traditional way

You have an application. You want it to run on a server.

1. Get a server
2. Install Python on it
3. Install the libraries your code needs
4. Install the system packages *those* libraries need
5. Copy your code across
6. Hope that the versions match what you had locally

Every step is manual, and nothing records what you did.

The containerization way

Containerization is packaging an application together with everything it needs to run, into a single portable unit called a **container**.

Inside the container:

- Your code
- The libraries it needs
- The system tools and settings it needs

You move that container around. It behaves the same everywhere.

Why it helps

- **Isolation:** each container is independent. Two projects needing two different Python versions stop being a problem.
- **Portability:** same behaviour on your laptop, on your colleague's, on a server, in CI.
- **Efficiency:** lightweight, starts in seconds.
- **Consistency:** no environment differences between development and production.

Think of shipping containers

Before: every cargo was loaded by hand, in its own shape and size.

Then someone standardised the **box**.

- One standard format, whatever is inside
- Moves between ships, trucks and trains without being unpacked
- Contents stay isolated and protected
- Easy to stack, count and manage

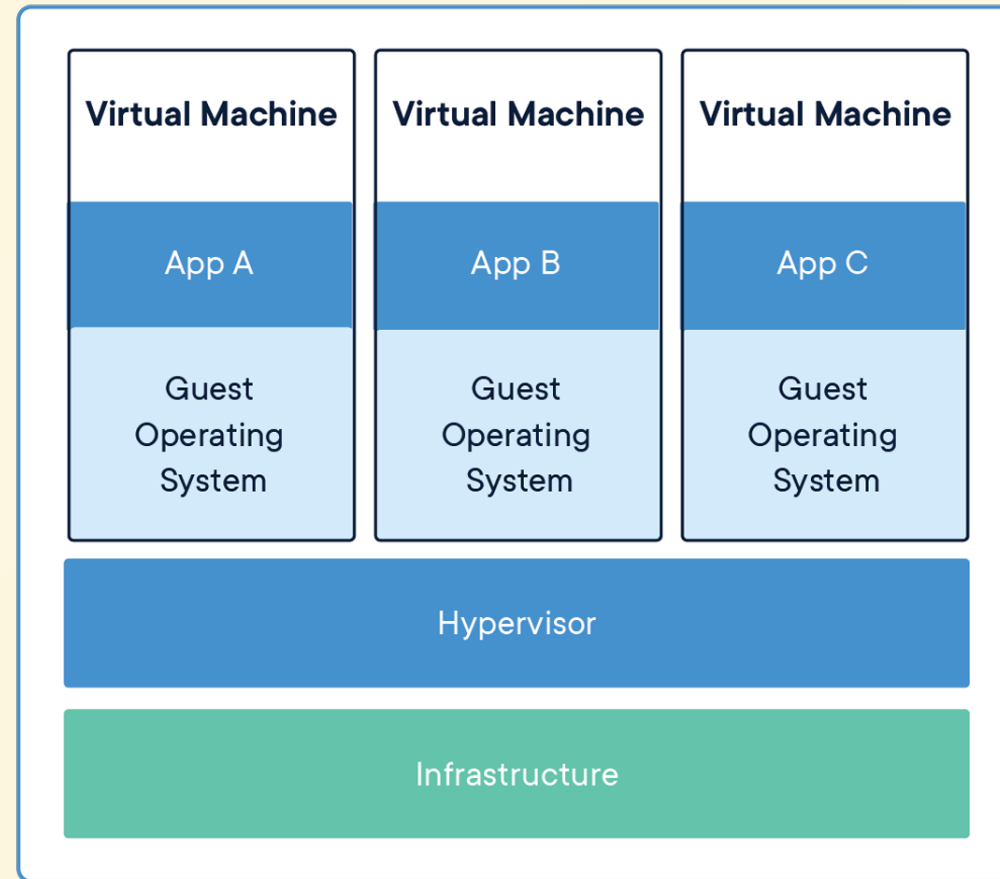
The ship does not need to know what is in the box. That is the whole idea, and where the name comes from.

Two ways to package an environment

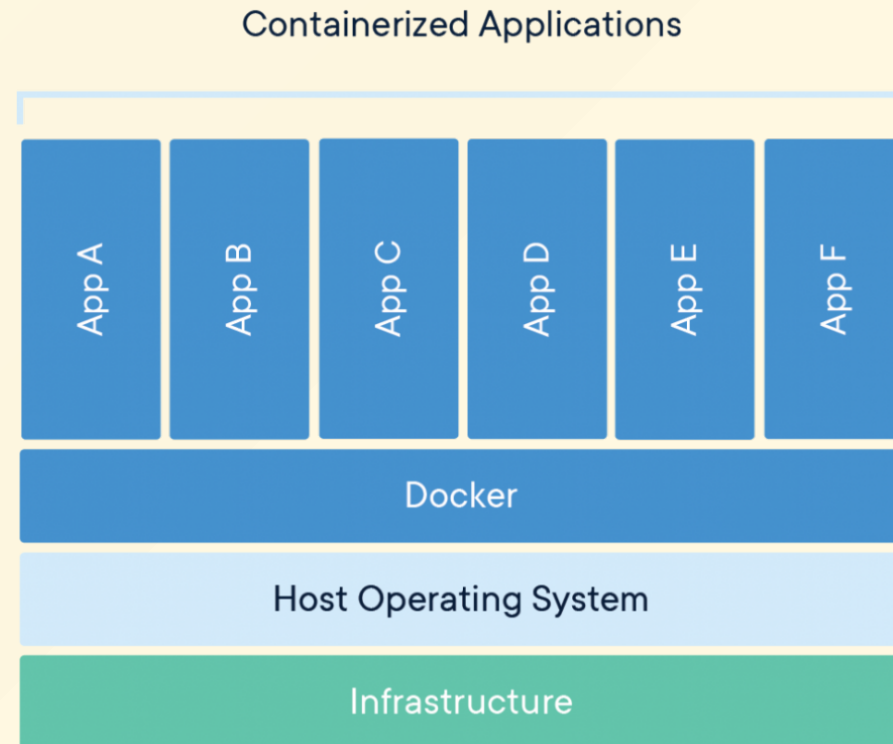
	Virtual Machines	Containers
Isolation	Complete OS	Process-level
Size	GBs (entire OS)	MBs (app + deps)
Startup	Minutes	Seconds
Resources	Heavy, each VM a full OS	Light, shares host kernel

A VM virtualises the **hardware**. A container just isolates a **process**; it uses the host's kernel.

Virtual Machine



Container



What is Docker?

Docker is a platform that uses containerization to package applications and their dependencies into lightweight, portable containers.

- **A containerization platform:** package an app with everything it needs
- **Lightweight:** shares the host's kernel, far faster than a VM
- **Portable:** build once, run anywhere
- **Consistent:** the same environment from your laptop to production

It is not the only one, but it is the one everybody uses.

Four words you will hear all day

Dockerfile: a text file of instructions. The recipe.

Image: the built result. Read-only, versioned, shareable.

Container: a running instance of an image.

Registry: where images are stored and shared (Docker Hub, ghcr.io).

The next slides take them one at a time.

Dockerfile

A **Dockerfile** is a text file with the instructions to build an image. The recipe. This turns the env into something you can version control.

```
FROM python:3.14-slim
COPY . /app
RUN pip install -r /app/requirements.txt
CMD ["python", "/app/app.py"]
```

- It lives **in your repository**, next to your code
- It is reviewed in pull requests, like any other file
- Same Dockerfile → same image, for everyone

Image

An **image** is a blueprint. It is created by snapshotting the result of the filesystem after running each line of the Dockerfile.

- **Read-only**: it cannot change once built
- Contains your application code, its dependencies and the runtime
- **Versioned** with tags, so `python:3.14` and `python:3.9` are two different images
- **Shareable**: you can publish it and someone else can download it

You do not run an image. You create containers from it.

Container

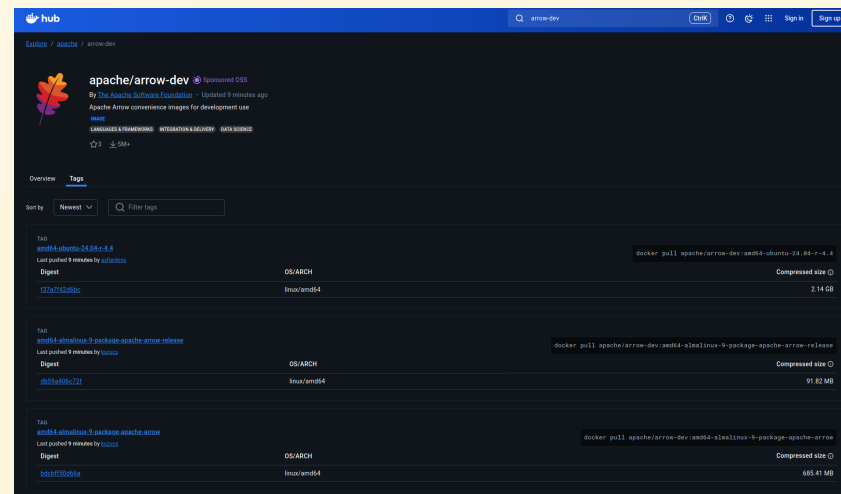
A **container** is a running instance of an image.

- Adds a thin **writable layer** on top of the read-only image
- **Isolated**: its own filesystem, network and processes
- **Ephemeral**: anything written inside is lost when it is removed
- You can start **many** containers from the same image

Created → Running → Stopped → Removed

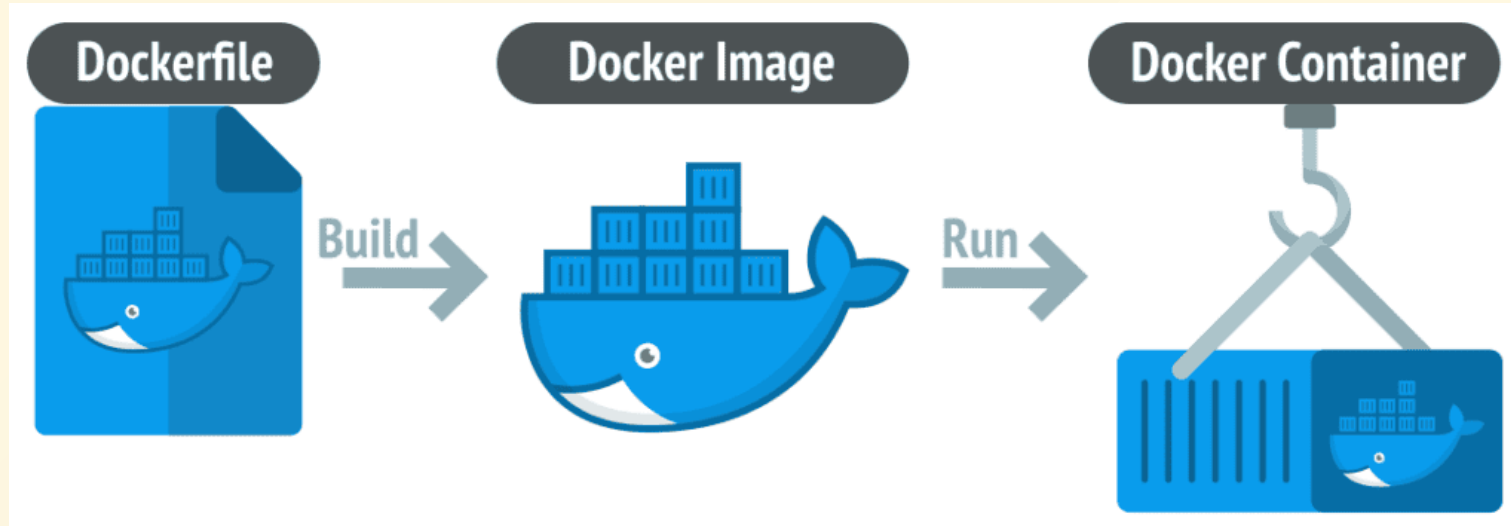
Registry

A **registry** stores and distributes images. Think of it as the GitHub of images: you **push** yours, you **pull** other people's.



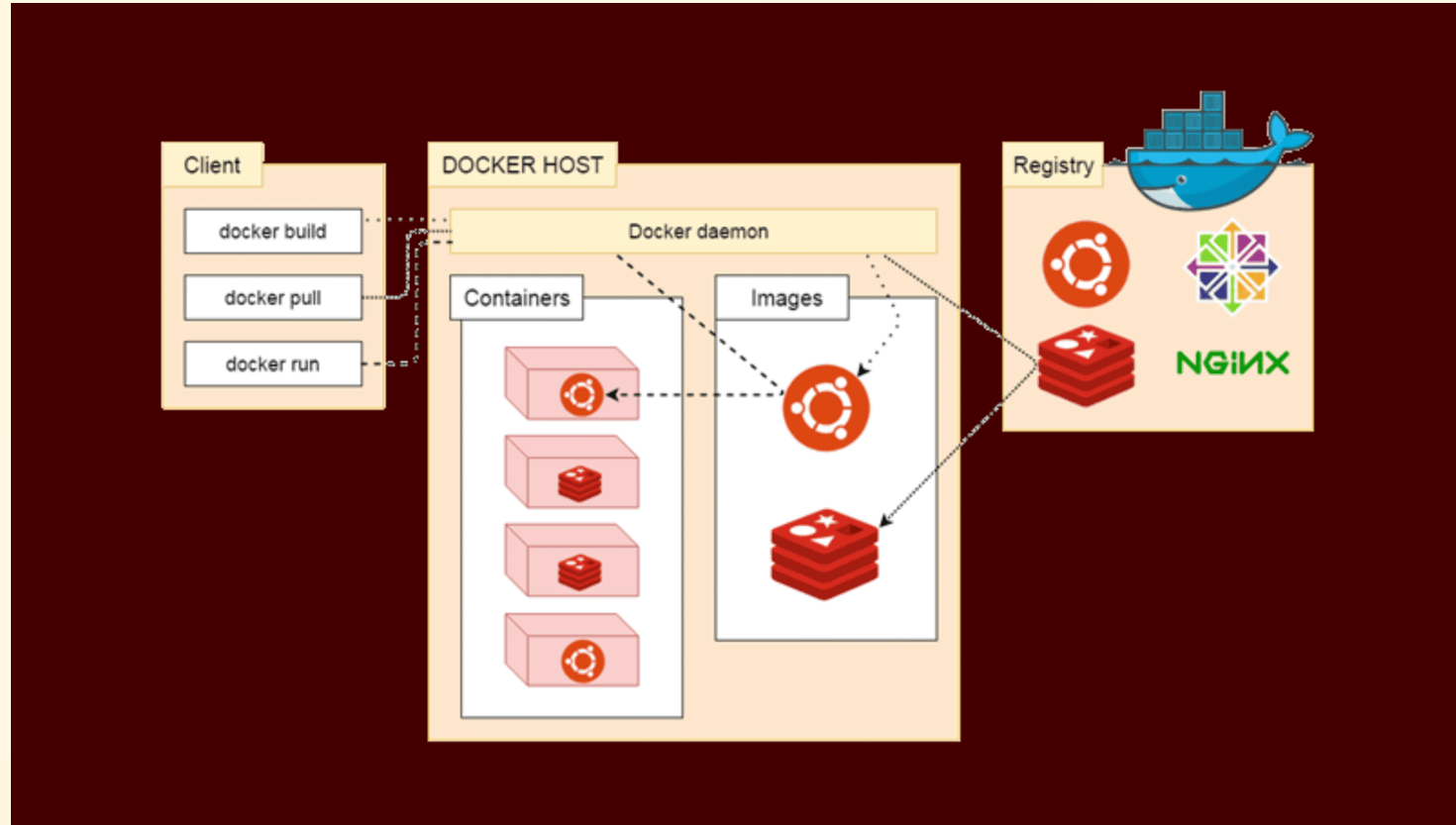
Docker Hub is the default public one. Others: GitHub Container Registry (**ghcr.io**), and one from each cloud provider.

Putting it together



You **build** a Dockerfile into an image, and **run** an image as a container.

Docker architecture



Docker architecture

- **Docker Client:** the `docker` command you type
- **Docker Daemon:** the background service that does the actual work: building images, starting containers
- **Images:** the read-only templates
- **Containers:** the running instances
- **Registry:** where images are stored and shared

You talk to the client. The client asks the daemon. The daemon pulls from the registry when it needs something it does not have.

Part 1

Running containers

Install Docker

Linux: Docker Engine:

<https://docs.docker.com/engine/install/>

macOS / Windows: Docker Desktop:

<https://docs.docker.com/get-started/get-docker/>

On Windows, use the **WSL2** backend when it asks.

Check it works:

```
docker --version  
docker info
```

Your first container

```
docker run hello-world
```

```
raulcd@rhoegar: ~ 81x29
raulcd@rhoegar:~ []$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:54e66cc1dd1fcb1c3c58bd8017914dbed8701e2d8c74d9262e26bd9cc1642d31
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```


What just happened

1. Docker looked for the `hello-world` image **locally**
2. Didn't find it, so **pulled** it from Docker Hub
3. **Created** a container from that image
4. **Ran** it and printed a message
5. The container **exited**, and is still there, stopped

```
docker ps          # nothing: no running containers
docker ps -a       # there it is, "Exited (0)"
```

An interactive container #1

```
docker run -it ubuntu bash
```

- `-i` keep STDIN open
- `-t` give me a terminal
- `ubuntu` the image
- `bash` the command to run inside

An interactive container #2

Inside, try:

```
cat /etc/os-release      # you are on Ubuntu  
ls /  
python3 --version        # not installed. It's a minimal image  
exit                     # Will go back to your laptop
```

A Python container

```
docker run -it python python
```

You get a Python 3.14 REPL (latest), without installing Python 3.14.

```
>>> import sys; sys.version  
>>> exit()
```

An old Python container

Same for any version:

```
docker run -it python:3.9 python
```

This alone is worth the class. Testing on another Python version is one command, not an afternoon.

Containers are ephemeral

```
docker run -it ubuntu bash
# inside:
echo "important results" > /results.txt
exit
```

```
docker run -it ubuntu bash
# inside:
cat /results.txt           # No such file or directory
```

The second `run` made a **new container** from the same image.

Anything you write inside a container is gone when the container is removed. We fix this with volumes, later.

Managing containers and images

```
docker ps                # running containers
docker ps -a             # all of them, including stopped
docker stop <name|id>    # stop a running one
docker rm <name|id>      # delete a stopped one
docker logs <name|id>    # what did it print?
docker exec -it <name> bash # shell into a RUNNING container

docker images            # local images
docker rmi <image>       # delete an image
```

```
docker run --name myapp ... gives it a name instead of
nostalgic_bohr.
```

Cleaning up

Docker will happily eat your disk.

```
docker system df      # how much am I using?  
docker container prune # remove all stopped containers  
docker image prune    # remove dangling images  
docker system prune -a # remove everything unused. Careful.
```

Run this every few weeks. Or discover it when your disk is full.

Exercise 1

1. Run `hello-world`
2. Run an interactive `ubuntu` container. Find its Ubuntu version
3. Run `python:3.9` and print `sys.version`
4. `docker ps -a`: how many containers do you have now?
5. Create a file inside a container, exit, start a new one, look for it
6. Clean up with `docker container prune`

Question: you ran three different images. Where did they come from, and where are they now?

Part 2

Building your own image

Why build your own

Running `python:3.14` is nice, but your project needs:

- Your code
- Your dependencies (`pandas`, `scikit-learn`, ...)
- Your configuration

You want **one artifact** that has all of it, that anyone can run.

That artifact is an image, and you describe it in a **Dockerfile**.

A Dockerfile

```
FROM python:3.14-slim      # base image
WORKDIR /app               # cd /app for everything below
COPY requirements.txt .    # copy from your machine into the image
RUN pip install -r requirements.txt # run a command at BUILD time
COPY . .                  # copy the rest of the code
CMD ["python", "app.py"]  # what to run at START time
```

Each instruction creates a layer. Layers are cached, which is what makes rebuilds fast. There is more on this in the extra material at the end.

The instructions you will actually use

FROM: the base image. Always first.

```
FROM python:3.14-slim
```

WORKDIR: working directory inside the image.

```
WORKDIR /app
```

COPY: From host to image.

```
COPY requirements.txt .  
COPY src/ /app/src/
```

The instructions you will actually use

RUN: execute at **build** time, result is baked into the image.

```
RUN pip install -r requirements.txt  
RUN apt-get update && apt-get install -y curl
```

ENV: environment variable.

```
ENV PYTHONUNBUFFERED=1
```

CMD: what runs at **start** time. Only the last one counts.

```
CMD ["python", "app.py"]
```

RUN vs CMD

The single most common confusion:

	When	How often
<code>RUN</code>	Build time, <code>docker build</code>	Once, result saved in the image
<code>CMD</code>	Start time, <code>docker run</code>	Every time you start a container

`RUN pip install pandas` → pandas is *in* the image.

`CMD ["python", "app.py"]` → the image doesn't run anything until you start a container.

Let's build one

Start your visual studio Code and create a folder called **my-app**.

Create three empty files with the following structure.

```
my-app/  
├── Dockerfile  
├── requirements.txt  
└── app.py
```


Let's build one

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Let's build one

requirements.txt

```
flask==3.1.3
```

Dockerfile

```
FROM python:3.14-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build it

```
docker build -t my-app .
```

- `-t my-app` tag (name) for the image
- `.` current directory for the **build context**: the directory sent to Docker

Run it

```
docker run -p 5000:5000 my-app
```

- `-p 5000:5000` = `host_port:container_port`

Open <http://localhost:5000>.

Stop it with `Ctrl+C`.

.dockerignore

`COPY . .` copies *everything* in the build context, including `.git`, your virtualenv and your 4 GB of data.

You can create a `.dockerignore` to have a smaller context and do not ship secrets or environment, same idea as `.gitignore`:

```
.git
__pycache__/
*.pyc
.venv/
data/
.env
*.ipynb_checkpoints
```

A few rules worth following

Pin your base image

```
FROM python:3.14-slim    # not python:latest
```

`latest` changes under you. Reproducibility is the whole point.

Prefer `-slim` images: `python:3.14` is ~1 GB, `python:3.14-slim` is ~150 MB.

Never put secrets in a Dockerfile. Every layer is readable by anyone who has the image. Pass them at runtime.

Exercise 2

Build the Flask app from the previous slides.

1. Create the three files
2. `docker build -t my-app .`
3. `docker run -p 5000:5000 my-app` and open it in a browser
4. Change the message in `app.py`, rebuild, and run it again
5. Add a `.dockerignore`

Part 3

Data, config and multiple containers

Getting data in and out

Containers are ephemeral. Your data should not be inside one.

Bind mount: a host directory, visible inside the container:

```
docker run -v $(pwd)/data:/app/data my-app
```

Best for development: edit files on your machine, the container sees them immediately.

Getting data in and out

Named volume: storage Docker manages:

```
docker volume create mydata  
docker run -v mydata:/app/data my-app
```

Best for databases and anything that must survive.

Configuration

Never hardcode a connection string or an API key.

```
docker run -e DEBUG=true -e API_KEY=secret my-app
```

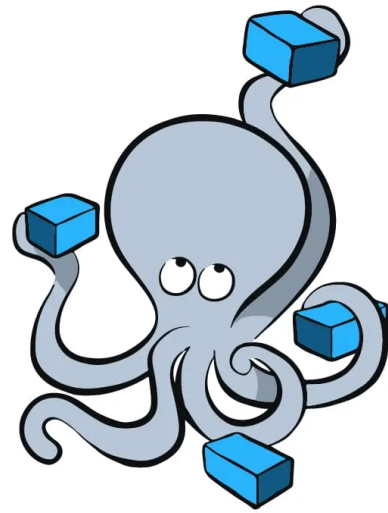
Or from a file:

```
# .env    (add it to .gitignore AND .dockerignore)
DEBUG=true
DATABASE_URL=postgresql://user:pass@db:5432/mydb

docker run --env-file .env my-app
```

Same image, different config per environment. That is the point.

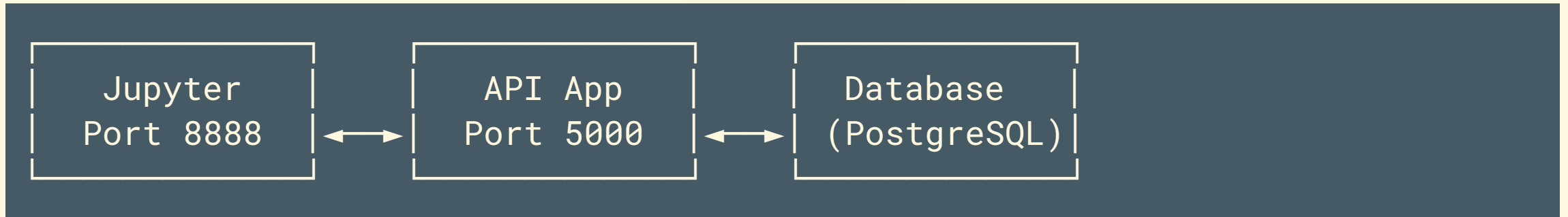
Docker Compose



docker
Compose

Why Compose

Your project is rarely one container:



Starting that by hand is three long `docker run` commands, in the right order, on the right network.

Compose is one YAML file and one command.

compose.yaml

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb
    depends_on:
      - db

  db:
    image: postgres:17
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Services talk to each other by name

Look at that connection string again:

```
postgresql://user:pass@db:5432/mydb
                ^ ^
```

`db` is the **service name** from `compose.yaml`.

```
conn = psycopg2.connect(
    host="db",           # not an IP address
    database="mydb",
    user="user",
    password="pass",
)
```

Compose commands

```
docker compose up           # start everything, logs in the terminal
docker compose up -d        # start in the background
docker compose ps           # what is running
docker compose logs -f web  # follow the logs of one service
docker compose down         # stop and remove
docker compose down -v      # ...and delete the volumes too
docker compose build         # rebuild images
```

Note: `docker compose` (with a space). The old `docker-compose` is the deprecated v1.

What this gets you

New person joins the project:

```
git clone <repo>  
docker compose up
```

That is the whole onboarding. No "install PostgreSQL 17, then..."

The environment is now **in the repository**, versioned with the code, reviewed in pull requests, identical for everyone.

This is what we meant last week by reproducibility.

Let's add a database

Back to `my-app`. We add one file and change two.

```
my-app/  
├── Dockerfile           (unchanged)  
├── requirements.txt      (one new line)  
├── app.py               (talks to the database now)  
└── compose.yaml         (new)
```

The app will count how many times the page has been visited, and store that count in PostgreSQL.

Let's add a database

requirements.txt: add one line.

```
flask==3.1.3  
psycopg2-binary==2.9.10
```

`psycopg2` is the PostgreSQL driver for Python. The `-binary` version ships precompiled, so you don't need a C compiler in your image.

Let's add a database

app.py

```
import os
import psycopg2
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    conn = psycopg2.connect(os.environ["DATABASE_URL"])
    with conn, conn.cursor() as cur:
        cur.execute("CREATE TABLE IF NOT EXISTS visits (id SERIAL PRIMARY KEY)")
        cur.execute("INSERT INTO visits DEFAULT VALUES")
        cur.execute("SELECT COUNT(*) FROM visits")
        count = cur.fetchone()[0]
    conn.close()
    return f"Hello from Docker! Visit number {count}"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Let's add a database

Two things to notice in that file:

1. The connection details come from the environment

```
psycopg2.connect(os.environ["DATABASE_URL"])
```

No password in the code. Compose passes it in.

2. There is no `localhost` anywhere

The host will be `db`, the service name. We set that in the next file.

Let's add a database

Create a new **compose.yaml** file.

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb
    depends_on:
      - db

  db:
    image: postgres:17
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Let's add a database

```
docker compose up
```

Compose will build your `web` image, pull `postgres:17`, create a network and a volume, and start both containers.

You will see the logs of the two services interleaved, with a colour each.

Open <http://localhost:5000> and refresh a few times. The counter goes up.

Exercise 3

Get the two-container setup running, then break it on purpose.

1. Create the four files from the previous slides
2. `docker compose up`, open the page, refresh until the counter is 5
3. In another terminal (2): `docker compose ps`: how many services?
4. Terminal 2: `docker compose logs db`: what did PostgreSQL print on startup?

Exercise 3

5. Type `docker compose down` on terminal 2, then `docker compose up` again on terminal 1. Refresh the page: **Is the counter still at 5?**
6. Now Terminal 2: `docker compose down -v`, then Terminal 1 `docker compose up`. **What is the counter now?**

Exercise 3: what step 5 and 6 show

`docker compose down` removes the **containers**. The named volume `pgdata` survives, so the database keeps its data.

`docker compose down -v` also removes the **volumes**. The data is gone for good.

This is the difference between "I restarted my environment" and "I deleted my database". Worth knowing which flag is which before you type it on something that matters.

Where this is going

- **Session 3 (Storage):** your data does not belong in the image. Where does it belong?
- **Session 4 (Cloud):** the same image, running on a machine that is not your laptop
- **Session 5 (MLOps):** reproducible training runs
- **Your project:** `compose.yaml` in the repo from sprint 1

If you get stuck

```
docker ps -a          # is it even running? why did it exit?  
docker logs <container> # what did it say before dying?  
docker exec -it <container> bash    # go inside and look around
```

Most Docker problems are one of:

- The port is not published (`-p`) or already in use
- The app listens on `127.0.0.1` instead of `0.0.0.0`
- The file you expected is not in the image (check `.dockerignore`)
- You rebuilt the image but are running the old container

**If you want to learn more about
Docker...**

Extra material

Extra: not required

You do not need any of this for the project.

You will need some of it the first time a container does something you did not expect.

CMD vs ENTRYPOINT

```
CMD ["python", "app.py"]
```

A **default**. `docker run myimage python other.py` overrides it.

```
ENTRYPOINT ["python", "app.py"]
```

Always runs. Arguments to `docker run` are appended to it.

CMD vs ENTRYPOINT

Common pattern: fixed program, default arguments:

```
ENTRYPOINT ["python", "train.py"]  
CMD ["--epochs", "10"]
```

```
docker run myimage --epochs 50 → python train.py --epochs 50
```


COPY vs ADD

Both copy files into the image.

ADD also:

- Extracts local tar archives automatically
- Can fetch a URL

```
ADD https://example.com/data.tar.gz /tmp/
```

Use COPY. **ADD**'s extra behaviour is surprising more often than it is useful. Use **RUN curl** when you need to download something, so it is explicit.

Layers and the build cache

Every instruction in a Dockerfile creates a **layer**. Change one line in `app.py` and rebuild: watch the output:

```
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install -r requirements.txt
=>          [5/5] COPY . .
```

`pip install` was **not** re-run. Docker reuses a layer when that instruction and its inputs have not changed.

Once one layer changes, **every layer after it is rebuilt**.

Why requirements come first

```
COPY requirements.txt .           # changes rarely  
RUN pip install -r requirements.txt  
COPY . .                         # changes every time you edit code
```

If you wrote `COPY . .` **before** `pip install`, every single code change would reinstall every dependency.

Order your Dockerfile from **least likely to change** to **most likely to change**. This is the difference between a 2-second and a 3-minute rebuild.

Multi-stage builds

Build in a big image, ship a small one. Compilers and build tools stay in the first stage. The shipped image is a fraction of the size:

```
FROM python:3.14 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt

FROM python:3.14-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
CMD ["python", "app.py"]
```

Sharing images

```
docker build -t <username>/my-app:v1.0 .  
docker push <username>/my-app:v1.0
```

anyone, anywhere

```
docker pull <username>/my-app:v1.0
```

Registries:

- Docker Hub: the default, public
- **GitHub Container Registry** (`ghcr.io`): free with your repo, and it sits right next to the code
- AWS ECR / Google Artifact Registry / Azure ACR

Compose: dev vs prod

```
docker compose -f compose.yaml -f compose.dev.yaml up
```

Later files override earlier ones, so `compose.dev.yaml` only contains the differences; a bind mount of your source, debug on, an extra port.

Useful service options not covered:

- `restart: always`
- `healthcheck:` and `depends_on: condition: service_healthy`
- `profiles:` services that only start when asked for

Beyond one machine

Compose runs containers on **one** machine. When you need more:

- **Kubernetes:** the standard for orchestrating containers across a cluster. Scheduling, scaling, self-healing, rolling deploys.
- **Managed services:** AWS ECS/Fargate, Google Cloud Run, Azure Container Apps. Hand them an image, they run it.

Cloud Run and Fargate take exactly the image you built today.
We look at this in session 4.

Where to learn more

- <https://docs.docker.com/get-started/>: the official guide is good
- <https://docs.docker.com/reference/dockerfile/>: every instruction
- <https://github.com/hadolint/hadolint>: a linter for Dockerfiles
- <https://docs.docker.com/build/cache/>: if builds feel slow

Questions?

See you on 05/10 with Storage